MODULE 3

PYTHON PROGRAMMING

1BPLC105B / 205B

DICTIONARIES • NUMPY • FILES • QUESTION BANKS

DICTIONARIES → NUMPY → FILES → PRACTICE

What's inside

Part A — Dictionaries
Operations, methods, counting, aliasing and copying, nested dictionaries

Part B — NumPy
Arrays, shape, dtype, slicing, masking, broadcasting

Part C — Files
Read, write, append, binary files, directories, fetching from the Web

Part D — Practice & Revision
Common errors, practice questions, quick revision table

Question banks — Modules 1 to 5

5-mark questions
10 per module • definitions, short programs

6-mark questions
8 per module • explanation with example or trace

8-mark questions
6 per module • detailed answers and lab programs

Syllabus-based
Follows the 1BPLC105B / 205B module topics and lab list

Course Notes

Prof. Kiran Mudaraddi

Learn the idea • Visualize the logic • Write the code • Test the result

MODULE 3 — Dictionaries • NumPy • Files

Module 3 — Contents

- **Part A — Dictionaries:** dictionary operations, dictionary methods, aliasing and copying
- **Part B — NumPy:** about, shape, slicing, masking, broadcasting, dtype
- **Part C — Files:** about files, writing our first file, reading line-at-a-time, turning a file into a list of lines, reading the whole file at once, binary files, directories, fetching something from the Web
- **Part D — Practice & Revision:** common errors, short practice questions, quick revision table

Module 3 — Learning Outcomes

After studying this module, students should be able to:

- Create dictionaries and use key–value pairs to store, look up, update and delete data.
- Use dictionary methods (keys, values, items, get, pop, update, setdefault, copy) and traverse a dictionary.
- Explain aliasing and copying of dictionaries, including the shallow-copy trap.
- Create NumPy arrays and explain shape, dtype, slicing, boolean masking and broadcasting.
- Write, read (line-at-a-time, whole file, list of lines) and append to text files; work with binary files and directories.
- Fetch data from the Web and process it with file-style code.

How to use these notes: Read the explanation first. Then run the example, compare the output, change one value, and run it again. Indentation in Python is part of the program.

PART A — DICTIONARIES

1. What is a Dictionary?

A **dictionary** (`dict`) is a **mutable** collection of **key : value** pairs. Lists use a position (index 0, 1, 2…) to find an item; a dictionary uses a **key** (a name, a word, an ID). It is called a *mapping* type: each key maps to one value.

Syntax: `dict_name = {key1: value1, key2: value2, ...}`

PYTHON

```
student = {"name": "Kiran", "age": 35, "branch": "CSE"}
print(student)
empty = {}
print(type(empty).__name__)
```

OUTPUT

```
{'name': 'Kiran', 'age': 35, 'branch': 'CSE'}
dict
```

Remember

- Curly braces `{ }` create a dictionary. `{}` is an **empty dictionary**, not an empty set.
- Keys must be **unique** and **immutable** (hashable): strings, numbers and tuples of immutables are allowed; a list cannot be a key.
- Values can be of any type, including lists and other dictionaries.
- From Python 3.7 onwards a dictionary **remembers insertion order**.

Feature	List	Dictionary
Accessed by	position (index)	key
Syntax	`[1, 2, 3]`	`{"a": 1, "b": 2}`
Mutable?	Yes	Yes
Duplicates	Values may repeat	Keys are unique
Best for	ordered sequences	lookup tables, counting, records

2. Accessing Values

Use the key inside square brackets. A missing key raises a `KeyError`. The `get()` method returns `None` (or a default you supply) instead of an error.

```python
student = {"name": "Kiran", "age": 35, "branch": "CSE"}
print(student["name"])
print(student.get("phone"))
print(student.get("phone", "N/A"))
# print(student["phone"])   # KeyError if uncommented
```

```
Kiran
None
N/A
```

3. Dictionary Operations

Adding, updating and deleting items:

```python
d = {"a": 1, "b": 2}
d["c"] = 3        # add a new key
d["a"] = 10       # update an existing key
del d["b"]        # delete a key
print(d)
```

```
{'a': 10, 'c': 3}
```

Assignment to a key that exists **updates** it; assignment to a new key **adds** it.

Length and membership: `len()` gives the number of pairs. The `in` and `not in` operators test **keys**, not values.

```python
d = {"a": 10, "c": 3}
print(len(d))
print("a" in d)
print(10 in d)
print(10 in d.values())
```

```
2
True
False
True
```

Traversal: looping over a dictionary gives its keys. Use `.items()` to get key and value together.

```python
marks = {"Maths": 85, "Physics": 78, "Chem": 90}
for subject in marks:
    print(subject, marks[subject])
print("---")
for subject, score in marks.items():
    print(subject, "->", score)
```

OUTPUT

```
Maths 85
Physics 78
Chem 90
---
Maths -> 85
Physics -> 78
Chem -> 90
```

4. Dictionary Methods

Method	Purpose	Example
`keys()`	all keys	`d.keys()`
`values()`	all values	`d.values()`
`items()`	all (key, value) pairs	`d.items()`
`get(k, default)`	value for k, or default if absent	`d.get("x", 0)`
`pop(k)`	removes key k and returns its value	`d.pop("a")`
`popitem()`	removes and returns the last inserted pair	`d.popitem()`
`update(other)`	adds/overwrites pairs from another dictionary	`d.update({"z": 5})`
`setdefault(k, v)`	returns d[k]; if absent, inserts k with value v	`d.setdefault("n", 0)`
`copy()`	returns a shallow copy	`e = d.copy()`
`clear()`	removes all pairs	`d.clear()`

PYTHON

```python
marks = {"Maths": 85, "Physics": 78, "Chem": 90}
print(list(marks.keys()))
print(list(marks.values()))
print(list(marks.items()))
print(marks.pop("Physics"))
print(marks)
marks.update({"Bio": 70, "Maths": 88})
print(marks)
```

OUTPUT

```
['Maths', 'Physics', 'Chem']
[85, 78, 90]
[('Maths', 85), ('Physics', 78), ('Chem', 90)]
78
{'Maths': 85, 'Chem': 90}
{'Maths': 88, 'Chem': 90, 'Bio': 70}
```

5. Counting with a Dictionary (word frequency)

A dictionary is the natural tool for counting. The key is the item; the value is its count. This is the idea behind Lab Program 4 (10 most frequent words in a text file).

PYTHON

```python
text = "to be or not to be"
freq = {}
for w in text.split():
    freq[w] = freq.get(w, 0) + 1
print(freq)

ranked = sorted(freq.items(), key=lambda pair: pair[1], reverse=True)
print(ranked[:3])
```

OUTPUT

```
{'to': 2, 'be': 2, 'or': 1, 'not': 1}
[('to', 2), ('be', 2), ('or', 1)]
```

Trace: `freq.get(w, 0)` returns the current count (0 for a new word); adding 1 and storing it back updates the dictionary. `sorted(..., key=..., reverse=True)` orders pairs by count, highest first; slicing `[:3]` keeps the first three.

6. Aliasing and Copying

As with lists, assigning one dictionary variable to another does **not** copy the dictionary. Both names refer to the **same object** (aliasing). Use `copy()` to make a separate (shallow) copy.

PYTHON

```python
a = {"x": 1, "y": 2}
b = a            # alias: same dictionary
b["x"] = 99
print(a)

c = a.copy()     # clone: new dictionary
c["y"] = 0
print(a)
print(c)
```

OUTPUT

```
{'x': 99, 'y': 2}
{'x': 99, 'y': 2}
{'x': 99, 'y': 0}
```

Shallow copy trap: `copy()` copies only the outer dictionary. If a value is itself a list or dictionary, that inner object is still **shared** between the original and the copy.

```python
orig = {"name": "Amal", "marks": [80, 90]}
copy1 = orig.copy()
copy1["marks"].append(100)
print(orig["marks"])
```

OUTPUT

```
[80, 90, 100]
```

For a fully independent copy of nested data use `copy.deepcopy()` from the `copy` module (covered again in Module 5).

7. Nested Dictionaries

```python
class_data = {
    "A101": {"name": "Asha", "marks": 82},
    "A102": {"name": "Ravi", "marks": 91},
}
print(class_data["A102"]["name"])
for roll, info in class_data.items():
    print(roll, info["name"], info["marks"])
```

OUTPUT

```
Ravi
A101 Asha 82
A102 Ravi 91
```

PART B — NUMPY

8. About NumPy

NumPy (Numerical Python) is a library for fast numerical computing. Its core object is the **ndarray** — an N-dimensional array that holds items of **one data type** in a compact block of memory. Because operations run on the whole array at once (called **vectorization**), NumPy code is shorter and much faster than looping over lists.

Import it with the standard alias:

```python
import numpy as np
```

Feature	Python list	NumPy array
Element types	Mixed allowed	One dtype for all items
`x * 2`	Repeats the list	Multiplies every element
Speed on large data	Slower	Much faster
Multi-dimensional maths	Awkward (nested lists)	Built in

```python
import numpy as np
print([1, 2, 3] * 2)
print(np.array([1, 2, 3]) * 2)
```

```
[1, 2, 3, 1, 2, 3]
[2 4 6]
```

9. Creating Arrays

```python
import numpy as np
a = np.array([10, 20, 30, 40])
print(a)
print(type(a).__name__)
print(np.zeros((2, 3)))
print(np.ones(3))
print(np.arange(0, 10, 2))
print(np.linspace(0, 1, 5))
```

OUTPUT

```
[10 20 30 40]
ndarray
[[0. 0. 0.]
 [0. 0. 0.]]
[1. 1. 1.]
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
```

- `np.array(list)` — array from a list (a list of lists gives a 2-D array).
- `np.zeros(shape)`, `np.ones(shape)` — filled with 0 or 1 (floats by default).
- `np.arange(start, stop, step)` — like `range()`, but returns an array and allows float steps.
- `np.linspace(start, stop, n)` — `n` evenly spaced values, **including** the stop value.

10. Shape, ndim and size

The **shape** of an array is a tuple giving the length along each dimension: `(rows, columns)` for a 2-D array.

PYTHON

```python
import numpy as np
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.shape)
print(m.ndim)
print(m.size)
print(m.reshape(3, 2))
print(m.T)
```

OUTPUT

```
(2, 3)
2
6
[[1 2]
 [3 4]
 [5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

Attribute / method	Meaning	Result for `m`
`m.shape`	size along each dimension	`(2, 3)`
`m.ndim`	number of dimensions	`2`
`m.size`	total number of elements	`6`
`m.reshape(3, 2)`	same data, new shape (element count must match)	3 rows × 2 columns
`m.T`	transpose (rows ↔ columns)	shape `(3, 2)`

11. Data Types (dtype)

Every array has one **dtype**. NumPy chooses it from the data: all integers give an integer dtype, and a mix of integers and floats is promoted to float. You can set or change it explicitly.

PYTHON

```python
import numpy as np
b = np.array([1.5, 2, 3])
print(b.dtype)
c = np.array([1, 2, 3], dtype=float)
print(c)
d = c.astype(np.int32)
print(d, d.dtype)
```

OUTPUT

```
float64
[1. 2. 3.]
[1 2 3] int32
```

dtype	Meaning
`int32` , `int64`	signed integers (the default integer size depends on the system)
`float32` , `float64`	decimal numbers (`float64` is the default float)
`bool`	True / False
`<U5` etc.	fixed-length Unicode text

`astype()` returns a **new** array of the requested dtype. Converting float to int **truncates** the decimal part.

12. Indexing and Slicing

Slicing follows the same `start:stop:step` rule as strings and lists (stop is excluded). For 2-D arrays give one slice per dimension, separated by a comma: `m[row_slice, column_slice]` .

PYTHON

```python
import numpy as np
a = np.array([10, 20, 30, 40, 50])
print(a[1:4])
print(a[::2])

m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(m[1, 2])
print(m[1])
print(m[:, 0])
print(m[0:2, 1:])
```

OUTPUT

```
[20 30 40]
[10 30 50]
6
[4 5 6]
[1 4 7]
[[2 3]
 [5 6]]
```

Important — a slice is a view, not a copy. Changing a slice changes the original array. Use `.copy()` for an independent array (compare with list slices in Module 2, which *do* create a new list).

PYTHON

```python
import numpy as np
a = np.array([1, 2, 3, 4])
s = a[1:3]
s[0] = 99
print(a)
t = a[1:3].copy()
t[0] = 0
print(a)
```

OUTPUT

```
[ 1 99  3  4]
[ 1 99  3  4]
```

13. Masking (Boolean Indexing)

A comparison on an array gives an array of True/False values — a **mask**. Using the mask as an index selects only the elements where it is True.

PYTHON

```python
import numpy as np
a = np.array([5, 12, 7, 20, 3])
mask = a > 6
print(mask)
print(a[mask])
print(a[(a > 4) & (a < 15)])
a[a < 6] = 0
print(a)
```

OUTPUT

```
[False  True  True  True False]
[12  7 20]
[ 5 12  7]
[ 0 12  7 20  0]
```

Remember

- Combine conditions with `&` (and), `|` (or), `~` (not) — **not** the words `and`, `or`, `not`.
- Put **each condition in parentheses**: `(a > 4) & (a < 15)`.
- A mask can also be used on the left of an assignment to change only the selected elements.

14. Broadcasting

Broadcasting is NumPy's rule for doing arithmetic between arrays of **different shapes**. The smaller array is virtually "stretched" to match the larger one — without actually copying data.

Rule: compare the shapes from the **rightmost** dimension to the left. Two dimensions are compatible if they are **equal**, or **one of them is 1**. If any pair is incompatible, NumPy raises a `ValueError`.

PYTHON

```
import numpy as np
a = np.array([1, 2, 3])
print(a + 10)

m = np.array([[1, 2, 3],
              [4, 5, 6]])
row = np.array([10, 20, 30])
col = np.array([[100],
                [200]])
print(m + row)
print(m + col)
```

OUTPUT

```
[11 12 13]
[[11 22 33]
 [14 25 36]]
[[101 102 103]
 [204 205 206]]
```

Operation	Shapes	Compatible?	Why
`a + 10`	`(3,)` and scalar	Yes	scalar is stretched to every element
`m + row`	`(2, 3)` and `(3,)`	Yes	trailing dimension 3 = 3; row repeats down the rows
`m + col`	`(2, 3)` and `(2, 1)`	Yes	dimension 1 stretches across the columns
`m + np.array([1, 2])`	`(2, 3)` and `(2,)`	**No**	rightmost dimensions 3 and 2 differ, neither is 1 → `ValueError`

15. Useful Array Functions

The `axis` argument chooses the direction: `axis=0` works **down the columns**, `axis=1` works **across the rows**.

PYTHON

```
import numpy as np
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.sum())
print(m.sum(axis=0))
print(m.sum(axis=1))
print(m.mean())
print(m.max(), m.min())

data = np.array([10, 20, 30])
print(np.mean(data))
print(np.var(data))
print(np.std(data))
```

OUTPUT

```
21
[5 7 9]
[ 6 15]
3.5
6 1
20.0
66.66666666666667
8.16496580927726
```

The last three lines match the mean, variance and standard deviation program from Module 1 — NumPy does it in one call each (`np.var` and `np.std` use the population formula by default).

PART C — FILES

16. About Files

Variables vanish when a program ends. A **file** is a named location on disk that stores data **permanently**. Python can create files, write to them, read from them and append to them.

Every file operation follows three steps: **open → read/write → close.**

PYTHON

```
f = open("filename", "mode")   # open
# ... read or write ...
f.close()                       # close
```

Mode	Meaning	If file exists	If file does not exist
`"r"`	read (default)	opens it	`FileNotFoundError`
`"w"`	write	**erases** old content	creates it
`"a"`	append	adds at the end	creates it
`"rb"`, `"wb"`	read / write **binary**	as above	as above

Text files store characters (`.txt` , `.csv` , `.py`). **Binary files** store raw bytes (images, audio, `.pdf` , `.exe`).

Best practice — the `with` statement. `with open(...) as f:` closes the file automatically, even if an error occurs, so you cannot forget `f.close()` . All examples below use it.

17. Writing Our First File

`write()` takes a **string**. It does not add a newline for you, so put `"\n"` at the end of each line. Convert numbers with `str()` .

PYTHON

```
with open("test.txt", "w") as f:
    f.write("Hello, world!\n")
    f.write("Second line\n")
    f.write(str(25) + "\n")
print("File written")
```

OUTPUT

```
File written
```

Contents of `test.txt` :

FILE

```
Hello, world!
Second line
25
```

Appending: mode `"a"` adds to the end without erasing.

PYTHON

```
with open("test.txt", "a") as f:
    f.write("Appended line\n")
```

Warning: opening an existing file with `"w"` **deletes its old content** immediately.

18. Reading a File Line-at-a-Time

A file object is **iterable**: a `for` loop gives one line per iteration. Each line still ends with `"\n"`, so use `strip()` (or `print(line, end="")`) to avoid blank lines in the output.

Assume `test.txt` holds the three lines from Section 17.

PYTHON

```
with open("test.txt", "r") as f:
    for line in f:
        print(line.strip())
```

OUTPUT

```
Hello, world!
Second line
25
```

The `readline()` method reads **one** line each time it is called, returning an empty string `''` at the end of the file.

PYTHON

```
with open("test.txt") as f:
    print(repr(f.readline()))
    print(repr(f.readline()))
```

OUTPUT

```
'Hello, world!\n'
'Second line\n'
```

19. Turning a File into a List of Lines

`readlines()` returns **a list** with one string per line (newlines included). Use `strip()` on each item to clean them.

```python
with open("test.txt") as f:
    lines = f.readlines()
print(lines)
clean = [ln.strip() for ln in lines]
print(clean)
print(len(clean))
```

```
['Hello, world!\n', 'Second line\n', '25\n']
['Hello, world!', 'Second line', '25']
3
```

20. Reading the Whole File at Once

`read()` returns the **entire contents as one string**. `read(n)` returns the next `n` characters.

```python
with open("test.txt") as f:
    content = f.read()
print(content)
print(len(content))
print(content.split())
```

```
Hello, world!
Second line
25

29
['Hello,', 'world!', 'Second', 'line', '25']
```

(The blank line appears because `content` ends with `"\n"` and `print` adds another. The length 29 counts 14 + 12 + 3 characters including the newlines.)

Method	Returns	Use when
`for line in f`	one line at a time	large files, line-by-line processing
`f.readline()`	the next single line	reading a header or a few lines
`f.readlines()`	list of all lines	you need the lines as a list (e.g., to sort)
`f.read()`	one string of everything	small files, word counting, searching

21. Typical File Programs

(a) Sort the contents of a file and write them to another file (Lab Program 6). If `input.txt` contains `banana`, `apple`, `cherry` on separate lines:

```python
with open("input.txt") as f:
    lines = []
    for line in f.readlines():
        lines.append(line.strip())
lines.sort()
with open("sorted.txt", "w") as out:
    for ln in lines:
        out.write(ln + "\n")
print(lines)
```

```
['apple', 'banana', 'cherry']
```

(b) 10 most frequent words in a text file (Lab Program 4) — combines files, strings and dictionaries.

```python
with open("text.txt") as f:
    words = f.read().lower().split()

freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1

top10 = sorted(freq.items(), key=lambda pair: pair[1], reverse=True)[:10]
for word, count in top10:
    print(word, count)
```

For a file containing `to be or not to be`, the output is:

```
to 2
be 2
or 1
not 1
```

(c) Handling a missing file. Opening a file that does not exist raises `FileNotFoundError`. A `try / except` block (covered fully in Module 5) lets the program respond politely.

```python
try:
    with open("missing.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("File not found")
```

```
File not found
```

22. Working with Binary Files

Binary files are opened with `"rb"` / `"wb"`. Reading returns **bytes** objects, not strings. Use binary mode for images, audio and any non-text data.

PYTHON

```
data = bytes([72, 101, 108, 108, 111])
with open("data.bin", "wb") as f:
    f.write(data)

with open("data.bin", "rb") as f:
    content = f.read()
print(content)
print(list(content))
```

OUTPUT

```
b'Hello'
[72, 101, 108, 108, 111]
```

Copying any file (e.g., a photo) exactly:

PYTHON

```
with open("photo.jpg", "rb") as src, open("copy.jpg", "wb") as dst:
    dst.write(src.read())
```

Saving Python objects with `pickle`: the `pickle` module converts lists, dictionaries and other objects to bytes and back.

PYTHON

```
import pickle
record = {"name": "Asha", "marks": [80, 90]}
with open("record.pkl", "wb") as f:
    pickle.dump(record, f)
with open("record.pkl", "rb") as f:
    loaded = pickle.load(f)
print(loaded)
```

OUTPUT

```
{'name': 'Asha', 'marks': [80, 90]}
```

Caution: load pickle files only from sources you trust; unpickling untrusted data can run harmful code.

23. Directories

A **directory** (folder) holds files and other directories. The `os` module lets a program work with them.

Function	Purpose
`os.getcwd()`	current working directory
`os.listdir(path)`	list of names inside a directory

Function	Purpose
`os.mkdir(name)`	create a new directory
`os.path.exists(p)`	does the path exist?
`os.path.isfile(p)` / `os.path.isdir(p)`	is it a file / a directory?
`os.path.join(a, b)`	build a path correctly for the operating system
`os.walk(path)`	visit a directory and **all** its subdirectories

PYTHON

```python
import os
os.mkdir("demo")
print(os.path.exists("demo"))
print(os.path.isdir("demo"))
print(os.listdir("demo"))
```

OUTPUT

```
True
True
[]
```

Display a folder recursively (Lab Program 12). Suppose `demo` contains `a.txt` and a sub-folder `docs` that contains `b.pdf`. The function calls **itself** for every sub-folder.

PYTHON

```python
import os

def show(path, indent=0):
    for name in sorted(os.listdir(path)):
        full = os.path.join(path, name)
        if os.path.isdir(full):
            print(" " * indent + name + "  [folder]")
            show(full, indent + 4)
        else:
            print(" " * indent + name + "  [file]")

show("demo")
```

OUTPUT

```
a.txt  [file]
docs  [folder]
    b.pdf  [file]
```

Same task with `os.walk()`:

PYTHON

```python
import os
for folder, subfolders, files in os.walk("demo"):
    for name in files:
        print(os.path.join(folder, name))
```

`os.walk` gives, for each directory it visits, the folder path, the list of its sub-folders and the list of its files.

24. Fetching Something from the Web

The `urllib.request` module opens a web address much like a file. `urlopen()` returns a file-like object; `read()` gives **bytes**, so `decode()` converts them to a string.

PYTHON

```
import urllib.request

url = "https://www.example.com"
with urllib.request.urlopen(url) as response:
    html = response.read().decode("utf-8")

print(len(html) > 0)
print(html[:15])
```

Output (needs an internet connection; the text depends on the page):

OUTPUT

```
True
<!doctype html>
```

Download and save to a file:

PYTHON

```
import urllib.request
urllib.request.urlretrieve("https://www.example.com", "page.html")
```

Once the text is in a variable or a file, everything in this module applies: `split()` for words, a dictionary for frequencies, `readlines()` for lines.

> **Remember:** web access can fail (no internet, wrong address). Wrap it in `try / except`. The third-party `requests` library (`pip install requests`) is a popular alternative to `urllib`.

PART D — MODULE 3 PRACTICE & REVISION

25. Common Errors to Check

Error or misconception	What to remember
`d["x"]` on a missing key	Raises `KeyError`; use `d.get("x")` or test `"x" in d` first.
Using a list as a dictionary key	Keys must be immutable (hashable); use a tuple instead.
`10 in d` to search values	`in` checks **keys**; use `10 in d.values()`.
`b = a` to copy a dictionary	Creates an alias; use `a.copy()`.
`copy()` for nested data	Shallow copy; inner lists/dicts are still shared.
`{}` as an empty set	`{}` is an empty **dictionary**; use `set()` for an empty set.
`a and b` / `a or b` on arrays	Use `&`, `|`, `~` with parentheses.
Expecting a NumPy slice to be a copy	A slice is a **view**; use `.copy()`.
Adding arrays of incompatible shapes	`ValueError`; check the broadcasting rule (equal or 1).
Writing a number with `f.write(25)`	`write()` needs a string: `f.write(str(25))`.
Forgetting `"\n"` when writing lines	All text ends up on one line.
Opening an existing file with `"w"`	Old content is erased; use `"a"` to append.
Printing lines read from a file	Lines end with `"\n"`; use `strip()`.
Reading an image in text mode	Use `"rb"` / `"wb"` for binary files.
Forgetting to close a file	Use `with open(...)` so it closes automatically.

26. Short Practice Questions

1. Create a dictionary of 5 students (roll number → name) and print every pair.
2. Write a program to count the frequency of each character in a string using a dictionary.
3. Write a program to merge two dictionaries using `update()`.
4. Write a program that inverts a dictionary (values become keys). Assume values are unique.
5. Demonstrate aliasing and copying of a dictionary with a short program.
6. Create a NumPy array of numbers 1 to 10 and print only the even numbers using a mask.

7. Create a 3 × 3 array with `arange` and `reshape`, then print its second column.
8. Add a 1-D array of shape `(3,)` to a 2-D array of shape `(2, 3)` and explain the result.
9. Compute mean, variance and standard deviation of 6 numbers using NumPy.
10. Write 5 names to a file, then read them back and print them in uppercase.
11. Read a text file and print the number of lines, words and characters.
12. Append a new line to an existing file and display the file again.
13. Copy a binary file to a new name.
14. Display all files with the extension `.txt` in the current directory.
15. Download a web page, save it to a file and count how many lines it has.

27. Quick Revision Table

Topic	Key takeaway
Dictionary	Mutable mapping of unique, immutable keys to values; `{k: v}`
`d[k]` vs `d.get(k)`	`KeyError` vs `None` / default
`in` on a dictionary	Tests keys only
`items()`	Gives (key, value) pairs for traversal
Aliasing vs copying	`b = a` shares; `a.copy()` is shallow
NumPy array	Fixed dtype, vectorized maths, `shape` / `ndim` / `size`
Slicing	`a[r1:r2, c1:c2]`; a slice is a **view**
Masking	`a[a > 5]`; combine with `&`, `|`, `~`
Broadcasting	Align shapes from the right; dimensions equal or 1
dtype	`astype()` returns a new array of another type
File modes	`r` read, `w` write (erases), `a` append, `b` binary
Reading	`for line in f`, `readline()`, `readlines()`, `read()`
`with open(...)`	Closes the file automatically
`os` module	`listdir`, `mkdir`, `path.exists`, `path.join`, `walk`
`urllib.request`	`urlopen()` / `urlretrieve()` fetch web content

Final revision strategy: For every example, predict the output, run the code, explain each line, then change one input and explain how the output changes.

Reference: These notes follow the Module 3 topics in the syllabus for 1BPLC105B / 205B. Recommended textbook: Peter Wentworth, Jeffrey Elkner, Allen B. Downey and Chris Meyers, *How to Think Like a Computer Scientist: Learning with Python 3*, 3rd Edition (2020). Practical reference: Al Sweigart, *Automate the Boring Stuff with Python*, 2nd Edition (2020).

QUESTION BANKS — MODULES 1 TO 5

Based on the syllabus for 1BPLC105B / 205B. Questions are grouped by marks: **5 marks** (short answer or short program), **6 marks** (explanation with example or output tracing), **8 marks** (descriptive answer or complete program). Use them for CIE tests, SEE preparation and viva practice.

Question Bank — Module 1

Syllabus: The way of the program • Variables, expressions and statements • Iteration • Functions with arguments and return values

5-Mark Questions

1. Define a program. Explain the basic building blocks of a program.
2. Differentiate between a compiler and an interpreter. How does Python execute a program (source → bytecode → PVM)?
3. Explain the immediate (interactive) mode and the script mode of the Python interpreter with their advantages and limitations.
4. What is debugging? Explain syntax error, runtime error and semantic error with one example each.
5. State the rules for naming variables in Python. Give two valid and two invalid names with reasons.
6. Explain the type converter functions `int()`, `float()` and `str()` with examples. What is the result of `int(-3.999)` and `int('12xyz')`?
7. Explain the order of operations in Python. Evaluate `5 + 4 * 3`, `2 ** 3 ** 2` and `(2 ** 3) ** 2`.
8. Explain concatenation and repetition operations on strings. Why do `"15" + 2` and `"Hello" - 1` give errors?
9. Write a Python program to read the total number of seconds and convert it into hours, minutes and seconds using the modulus operator.
10. Differentiate between `break`, `continue` and `pass` with a small example for each.

6-Mark Questions

1. Explain experimental debugging as an iterative cycle. Mention any four practical debugging tips.
2. Explain the `range()` function with its three forms. Write the sequences generated by `range(5)`, `range(2, 6)`, `range(1, 10, 2)` and `range(10, 0, -2)`.
3. Explain the working of a `while` loop step by step. Write a program to find the sum of the first N natural numbers using a `while` loop.

4. Write a program to generate the Collatz 3n + 1 sequence for a given n and count the steps. Trace the program for n = 6.

5. Explain arithmetic, comparison and logical operators in Python with a table and one example each.

6. Write a Python program to read two numbers and perform addition, subtraction, multiplication or division based on the user's choice (1-Add, 2-Subtract, 3-Multiply, 4-Divide) using a function.

7. Write a program to generate the first N terms of the Fibonacci sequence. Trace the values of `a` and `b` for N = 7.

8. Explain paired data and nested loops for nested data. Write a program that counts how many students take "CompSci" from a list of (name, subjects) tuples.

8-Mark Questions

1. Explain the three problem-solving tools — algorithm, flowchart and pseudocode — with the advantages and disadvantages of each. Write the algorithm and draw the flowchart to add two numbers.

2. Explain the types of programming languages (machine, assembly, high-level). Why are high-level languages preferred? Explain with a diagram how Python source code is executed.

3. Write Python programs to (a) read the name and year of birth of a person and display whether the person is a senior citizen, and (b) check whether a given year is a leap year. Explain the logic of each.

4. Explain functions with arguments and return values. Differentiate between parameters and arguments, and between fruitful and void functions. Write a function `find_average(n1, n2, n3)` and call it from a main program. List the advantages of functions.

5. Explain values, data types, variables and keywords. What is dynamic typing? Differentiate between mutable and immutable types with examples, and explain why `17 = n` is invalid.

6. Write Python programs to (a) check whether a number is prime, (b) reverse the digits of a number and (c) print a right-angled triangle star pattern using nested loops. Explain the logic of each.

Question Bank — Module 2

Syllabus: Strings • Tuples • Lists

5-Mark Questions

1. Explain string indexing (positive and negative) with an example. What happens if the index is out of range?
2. Explain string slicing. For `word = "Python"`, write the output of `word[0:4]`, `word[2:]`, `word[:3]` and `word[::2]`.
3. Why are strings immutable in Python? How can a string be "changed"? Illustrate with an example.
4. Explain the `in` and `not in` operators on strings with examples.
5. What are optional parameters? Explain with an example and state the rule about the order of default parameters.
6. Explain the string methods `split()`, `strip()`, `lstrip()` and `rstrip()` with examples.
7. Explain the string `format()` method with examples, including formatting a number to two decimal places.
8. Differentiate between a list and a tuple. Create a one-element tuple and explain the role of the comma.
9. Explain tuple assignment (unpacking) and show how two variables can be swapped without a temporary variable.
10. Explain aliasing and cloning of lists with an example. Why does `b = a` not create a copy?

6-Mark Questions

1. Write a user-defined function `find_letter(word, letter)` that returns the index of the first match or -1. Compare it with the built-in `find()` method, including the optional start argument.
2. Explain the looping-and-counting pattern. Write a program to count the number of vowels in a string.
3. Explain how a function can return multiple values using tuples. Write a function `divide(a, b)` that returns the quotient and remainder, and show how the caller unpacks them.

4. Explain composability of data structures with examples of a list of tuples, a tuple containing a list, and nested tuples. How are inner elements accessed?

5. Explain list slices, list mutability and list deletion with examples. Write the output of `nums = [10, 20, 30, 40, 50]` followed by `nums[1:4]`, `nums[::2]`, `nums[2] = 99` and `del nums[1]`.

6. Explain list parameters. Differentiate between a pure function and a modifier with one example each.

7. Explain nested lists and matrices. Write a program to read a 3 × 3 matrix and print the sum of each row.

8. Explain the relationship between strings and lists using `list()`, `split()`, `join()` and `list(range())` with examples.

8-Mark Questions

1. Explain string traversal using a `for` loop and a `while` loop, and string comparison using relational operators. Write a program to count the occurrences of a character and explain why `"Apple" < "banana"` is True but `"Python" == "python"` is False.

2. Write a Python program to create a list and perform the following operations: inserting an element, removing an element, appending an element, displaying the length, popping an element and clearing the list. Show the output at each step.

3. Write a program to read N numbers from the console, create a list, and print the mean, variance and standard deviation with suitable messages. Explain the formulas used.

4. Write a program to read a multi-digit number (as characters) from the console and print the frequency of each digit with a suitable message. Explain the logic.

5. Explain objects and references, aliasing and cloning of lists. Differentiate between shallow and deep copy with a nested list example. How does passing a list to a function allow the function to change the caller's list?

6. Write a program to read 6 subject marks of a student into a list and display the marks from the highest to the lowest score using the Bubble Sort technique. Explain how Bubble Sort works with a trace for a short list.

Question Bank — Module 3

Syllabus: Dictionaries • NumPy • Files

5-Mark Questions

1. Define a dictionary. Write its syntax with an example and list three differences between a list and a dictionary.
2. Explain how to add, update and delete items in a dictionary. How do `d[key]` and `d.get(key)` differ when the key is missing?
3. Explain any five dictionary methods (`keys` , `values` , `items` , `pop` , `update` , `setdefault` , `copy`) with examples.
4. Explain aliasing and copying of dictionaries. Why can `copy()` still share nested lists?
5. What is NumPy? List four advantages of NumPy arrays over Python lists.
6. Explain the `shape` , `ndim` , `size` and `dtype` attributes of a NumPy array with an example. How does `astype()` work?
7. Explain slicing of 1-D and 2-D NumPy arrays with examples. Is a NumPy slice a copy or a view?
8. List the file modes `r` , `w` , `a` , `rb` and `wb` with their meaning. What happens when a file opened with `"w"` already exists?
9. Differentiate between `read()` , `readline()` and `readlines()` with examples.
10. What is the `with` statement in file handling? Write a program to write three lines to a file and then append one more line.

6-Mark Questions

1. Write a program to count the frequency of each word in a sentence using a dictionary, and display the words in descending order of frequency.
2. Explain masking (boolean indexing) in NumPy. For `a = np.array([5, 12, 7, 20, 3])` , write the output of `a > 6` , `a[a > 6]` and `a[(a > 4) & (a < 15)]` .

3. Explain broadcasting with its rule. Show one compatible and one incompatible pair of shapes with examples.

4. Explain how to read a file line-at-a-time and how to turn a file into a list of lines. Write a program that prints every line of a file without the extra blank lines.

5. Write a program to read a text file and print the number of lines, words and characters.

6. Explain binary files. Write a program to copy a binary file, and explain the use of the `pickle` module.

7. Explain the `os` module functions used for directories: `getcwd`, `listdir`, `mkdir`, `path.exists`, `path.isdir`, `path.join` and `walk`. Write a program to list all files with the extension `.txt` in a folder.

8. Explain how to fetch a web page using `urllib.request` and save it to a file. Mention one error that can occur and how to handle it.

8-Mark Questions

1. Develop a program to print the 10 most frequently appearing words in a text file. Use a dictionary of distinct words and their frequencies, sort it in the reverse order of frequency and display the first 10 items. Explain each step.

2. Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. Use `strip()`, `len()`, `sort()`, `append()`, `open()`, `readlines()` and `write()`.

3. Explain NumPy in detail: creating arrays, shape, dtype, slicing, masking and broadcasting, with a suitable example for each.

4. Develop a program to display the contents of a folder recursively (name and type) when it has sub-folders and files. Explain the role of `os.listdir`, `os.path.isdir` and `os.walk`.

5. Develop a Student Grade Tracker: accept multiple students' names and marks, store them in a list of tuples or a dictionary, and display the average mark and the topper. Show a sample output.

6. Develop a Data Summary Generator: read a CSV file, convert it into dictionary form, and let the user run summary queries (maximum, minimum and average of a chosen column). Explain the approach.

Question Bank — Module 4

Syllabus: Modules • Namespaces and scope • Mutable versus immutable and aliasing • Classes and objects

5-Mark Questions

1. What is a module? Explain with an example how to import and use functions of the `math` module.
2. Explain any four functions of the `random` module (`random` , `randint` , `choice` , `shuffle`) with examples.
3. Explain the `time` module functions `time()` , `sleep()` and `ctime()` . How can `time()` be used to measure the running time of a program?
4. Explain the three import statement variants (`import m` , `from m import f` , `from m import *`) with examples and one drawback of each.
5. Explain how to create your own module and use it in another program.
6. What is a namespace? Name the kinds of namespaces in Python with an example of each.
7. Explain scope and the lookup rule (local → enclosing → global → built-in) with a short example.
8. Explain attributes and the dot operator with examples from a module and from an object.
9. Differentiate between mutable and immutable objects with examples. Why does `a = 5; a = 6` not change the integer 5?
10. Define a class and an object. Write the syntax of a class and create two objects of a class `Point` .

6-Mark Questions

1. Write a program using the `random` module to simulate rolling a die 20 times and count how many times each face appears.
2. Explain local and global variables with the `global` keyword. Predict and explain the output of a short program where a function assigns to a variable with the same name as a global variable.
3. Explain aliasing with mutable and immutable objects. Use `id()` and the `is` operator to show that two names refer to the same list but not to separate copies.

4. Define a class `Point` with attributes `x` and `y`. Create objects, assign and read attributes, and print them. Explain the role of `__init__` and `self`.

5. Explain how to add methods to a class. Add `distance_from_origin()` and `translate(dx, dy)` to the `Point` class and show their use.

6. Explain instances as arguments and parameters. Write a function `distance(p1, p2)` that takes two `Point` objects and returns the distance between them using the `math` module.

7. Explain instances as return values. Write a function `midpoint(p1, p2)` that returns a new `Point`.

8. Explain converting an instance to a string using the `__str__` method. Show the default output of `print(obj)` and the output after defining `__str__`.

8-Mark Questions

1. Create a module `mymath.py` with two functions and a variable. Write programs that use it with all three import variants, and explain how names are looked up in each case and how name conflicts can arise.

2. Explain mutable versus immutable objects and aliasing in detail. Show with programs what happens when integers, strings and lists are passed to a function that tries to change them.

3. Define a class `Rectangle` with attributes `width` and `height`. Add methods `area()`, `perimeter()` and `__str__()`. Create two objects, print their details and compare their areas.

4. Define a class `Point` with suitable methods. Write a function that takes a `Point` as a parameter and a function that returns a new `Point` as its result. Explain how instances are passed and returned.

5. Write a program using the `math` module to find the roots of a quadratic equation, handling real roots, equal roots and complex roots. Explain which functions of the module you used.

6. Explain scope and lookup rules (LEGB) in detail with a program that has built-in, global, enclosing and local variables. Explain how `global` and `nonlocal` change the behaviour.

Question Bank — Module 5

Syllabus: Objects are mutable, sameness, copying • Inheritance • Pure functions and modifiers • Generalization • Operator overloading • Polymorphism • Exceptions

5-Mark Questions

1. Explain with an example that objects are mutable. How does changing an attribute inside a function affect the caller's object?
2. Explain sameness in Python. Differentiate between `is` and `==`, and show how `__eq__` changes the behaviour of `==` for objects.
3. Differentiate between shallow copy and deep copy using `copy.copy()` and `copy.deepcopy()` with an example.
4. Differentiate between pure functions and modifiers with one example each.
5. What is inheritance? Write the syntax and a simple example of a derived class.
6. What is generalization? Explain with an example of moving common code into a base class.
7. What is operator overloading? List five special methods used for it (`__add__`, `__sub__`, `__mul__`, `__eq__`, `__lt__`, `__str__`) and their purpose.
8. Define polymorphism. Explain with an example of two classes that have a method with the same name.
9. What is an exception? List any five built-in exceptions and when each occurs.
10. Explain the `try` – `except` block with an example that handles `ZeroDivisionError` and `ValueError`.

6-Mark Questions

1. Explain copying of objects. Show an alias, a shallow copy and a deep copy of a `Rectangle` object that contains a corner `Point`, and explain the results of changing the corner.
2. Write a `Time` class with hours, minutes and seconds. Write a pure function `add_time(t1, t2)` and a modifier `increment(t, seconds)`, and explain the difference in behaviour.

3. Explain single inheritance and method overriding with an `Animal` base class and a `Dog` derived class. Explain the role of `super()`.

4. Explain operator overloading by implementing `__add__` for a `Point` class so that `p1 + p2` returns a new `Point`.

5. Explain polymorphism with a program in which objects of different classes are stored in a list and the same method is called on each.

6. Explain multiple `except` blocks, the `else` clause and the `finally` clause with an example.

7. Explain how to raise your own exceptions with the `raise` statement. Write a program that raises an exception when a negative age is entered.

8. Write a program that defines a custom exception class `InsufficientFundsError` and uses it in a `withdraw()` method of a `BankAccount` class. Show how it is caught.

8-Mark Questions

1. Define a class `Complex` to represent a complex number. Define a function that takes two `Complex` objects and returns a new `Complex` object with their sum. Write a program to read N (N ≥ 2) complex numbers and compute their sum.

2. Develop a function `DivExp(a, b)` that returns `c = a / b`. Write a suitable assertion for `a > 0` and raise an exception when `b = 0`. Write a program that reads two values from the console, calls `DivExp` and handles the exceptions.

3. Explain inheritance in detail: base and derived classes, overriding, `super()`, `isinstance()` and `issubclass()`. Write a program with a `Person` base class and a `Student` derived class.

4. Explain operator overloading in detail. Write a class `Vector` (or `Fraction`) that overloads `+`, `-`, `*`, `==` and `str()`, and demonstrate each operator.

5. Explain exception handling in detail: catching exceptions, multiple `except` blocks, `else`, `finally`, `raise` and user-defined exceptions. Write a program for a file-reading task that handles `FileNotFoundError` and `ValueError`.

6. Explain generalization and polymorphism with a program: a base class `Shape` with an `area()` method, derived classes `Circle` and `Rectangle`, and a loop that calls `area()` on a list of shapes.

Marks guide: 5 marks ≈ definition/explanation with one example (or a 6–8 line program) • 6 marks ≈ explanation with example and output/trace (or a 10–12 line program) • 8 marks ≈ detailed answer with diagram/table and complete program with sample output.